

ARM64 LINUX SERVICE ON EMU68 / PISTORM CLASSIC

Milestone - 31 August 2026 | Updated 1 September 2026

Fitz R/W, direct /dev/armblk0 persistence, ARMNET networking, and the sidecARM public PoC

Executive milestone

The project has now crossed two boundaries. First, Fitz/FUSE3 proved reliable bidirectional access from Alpine Linux to real AmigaOS files. Second, the persistence path has moved beyond the experimental Fitz + loop arrangement to a dedicated shared-memory block device: /dev/armblk0. AmigaOS remains the visible host environment under Emu68, while ARM64 Alpine runs on physical Cortex-A53 core 1. ARMNET remains the private IP link between the two operating systems. In the current test topology, Miami/MiamiDX still uses a 3Com EtherLink III PCMCIA card for normal LAN/Internet access, while armnet.device is a second private interface dedicated to sidecARM Linux.

The earlier Fitz experiment remains important because it validated the storage model: an ordinary file owned by AmigaOS can safely contain a native Linux ext2 filesystem. That experiment used linux-loop.img through Fitz and /dev/loop0 and survived format, write, sync, detach, reattach, remount, and SHA256 verification. The production direction is now cleaner: an AmigaOS-hosted file named armblk0.img is exposed directly to Linux as /dev/armblk0 through a purpose-built shared-memory block transport, eliminating FUSE, TCP, Fitz, and losetup from the private Linux persistence path.

Verified capability	Result
Direct Fitz filesystem R/W	Working
FUSE3 fitz-mount in Alpine initramfs	Working
AmigaOS file -> Linux loop device	Working
ext2 inside AmigaOS-hosted image	Persistent across detach/remount
Persistent apk --root tree	Working
Transparent system-wide persistence	Next step, not yet implemented

This update records the resulting two-path architecture: Fitz remains the interoperability filesystem for real AmigaOS files, while /dev/armblk0 is the private persistent Linux disk. It also records the preferred EMU68BOOT backing-file location, the observed SD0/boot-partition performance advantage, the current 3Com + ARMNET Miami topology, Windows routing, and the public proof-of-concept name sidecARM.

1. Stable platform inherited by today's work

The persistence milestone is meaningful because it was obtained without replacing the architecture built during the preceding week. The service remains a deliberately asymmetric dual-environment system: AmigaOS is still the visible host environment and Emu68 continues to execute the 68k machine, while Linux is a service operating system isolated on a normally unused ARM core. The goal is not to "boot Linux instead of AmigaOS"; it is to make modern Linux userspace and libraries available as a co-processor-like service while preserving the Amiga experience.

Strict Emu68 source baseline

All Emu68 replacement files in the milestone archive are tied to commit 9b4379a. This is a strict reproducibility requirement. The archive contains the modified CMakeLists.txt plus M68k_LINE4.c, armserve.c, emu68rom.c, mmu.c, and start.c. Applying the same files to an arbitrary newer or older Emu68 tree would mix assumptions about memory allocation, MOVEC communication, CPU startup, and the Zorro III aperture. Therefore 9b4379a plus the archived patch files should be treated as one indivisible baseline.

256 MiB service carveout

The known-good configuration reserves a contiguous 256 MiB physical block for the ARM service before later top-of-RAM allocations can fragment the region. The observed physical range in the baseline is 0x0c000000-0x1bffffff. The Amiga side sees a 256 MiB non-MEMLIST Zorro III aperture at 0x50000000-0x5ffffff. These are two aliases of the same backing memory, not two independent memories. Linux addresses the physical service region; AmigaOS accesses the Zorro III mapping.

A critical bridge fix remains the dedicated 32-bit `armservice_phys_base_68k` value. Emu68 runs AArch64 in a context where the original 64-bit `uintptr_t` alias was not suitable for the 68k MOVEC bridge and yielded zero. MOVEC 0x1e6 therefore returns the explicit 32-bit mirror. The Amiga Linux launcher obtains the service base through this mechanism and patches the Linux device tree dynamically.

Payload layout retained

```
+0x03b00000  ARMNET shared RAM
+0x04000000  Alpine initramfs in the larger Alpine launcher layout
+0x06000000  DTB in the Alpine launcher layout
```

The move to Alpine preserved the service-core design while giving the initramfs a much larger window. Nothing in today's FITZ or ext2 work requires a new physical memory model. This is important: persistence has been added at the filesystem/transport layer, not by making the Linux RAM carveout itself persistent.

2. CPU1 Linux, timer correctness, and ARMNET foundation

Linux executes on physical Cortex-A53 core 1, while Linux itself sees that processor as logical CPU0. That unusual mapping exposed one of the most important low-level problems in the project: the generic ARM timer counter advanced, but its `clockevent` interrupt never fired. The result was a kernel that could boot and run commands yet behaved as if time had stopped: jiffies did not advance and sleep, timeout, and repeated ping operations stalled.

Timer/IRQ root cause

The bcm2836 interrupt controller code indexes per-core registers using `smp_processor_id()`. In this proof-of-concept, Linux logical CPU0 corresponds to physical core1, so using logical index 0 selected the hardware bank for physical core0. The working fix intentionally hardcodes physical bank1 in the timer mask path, timer unmask path, and root interrupt handler. This is not presented as the final upstream-quality solution; it is the smallest verified correction for the logical/physical CPU-number mismatch.

With that workaround in place, sleep advances normally and loopback ping works, proving that the `clockevent` path is alive. Earlier experiments involving the BCM2835 system timer were diagnostic only and are explicitly not part of the clean fix. A future cleanup can derive physical affinity from `MPIDR_EL1` rather than hardcoding bank1, but the current patch is known-good and should remain untouched while higher layers are being stabilized.

ARMNET: private network between the two operating systems

ARMNET is the transport that made today's work possible. It is not TAP, a virtual Ethernet process, or a shared disk. It is a paired Linux/Amiga network driver using shared service RAM. On Linux, the built-in `armnet` driver maps normal cacheable shared memory relative to the physical service base. On AmigaOS, `armnet.device` finds the dedicated Zorro III service board and reaches the same shared rings through the Amiga aperture. The current ABI uses two sets of 64 slots with 2048-byte payloads plus metadata.

```
AmigaOS ARMNET / MiamiDX: 10.68.0.1
Linux arm0:                10.68.0.2/30
Current Amiga LAN:         3Com EtherLink III PCMCIA
```

The current Miami/MiamiDX configuration deliberately keeps the 3Com EtherLink III PCMCIA interface as the Amiga's normal physical LAN/Internet path. `armnet.device` is an additional private interface used only to reach Linux at 10.68.0.2. This keeps the proven Amiga network configuration intact while sidecARM develops. Linux-side ARMNET configuration is automatic in the current Alpine image.

Why ARMNET now has two distinct roles

ARMNET remains essential for operating-system interoperability: Fitz uses TCP over ARMNET so Linux can access real SYS:, Work:, RAM:, DEVS:, S:, C: and other AmigaDOS namespaces safely. Persistence no longer depends on this network path. /dev/armblk0 uses a separate direct shared-memory request/completion transport to an AmigaOS backing-file service. This separation is now a core design principle: Fitz for Amiga files; armblk for Linux storage.

3. Alpine service OS and consolidated runtime UX

The Linux userspace has evolved from a tiny BusyBox proof-of-concept into Alpine Linux 3.24.1 AArch64. This matters because persistence is useful only when there is meaningful state to preserve: packages, configuration files, scripts, databases, caches, service settings, and application data. Alpine provides a compact musl-based environment with apk package management while remaining small enough for the constrained PiStorm service-OS design.

Real init and terminal separation

BusyBox init is now PID 1. Earlier experiments had custom ArmTerm-related code acting too close to init responsibilities, which led to zombie ssl_client processes during network activity. The current architecture separates duties: standard init owns process lifecycle; armterm-relay only bridges a Linux PTY to shared memory; and the Amiga ArmTerm frontend is a terminal client. The POC15b frontend includes the stricter raw-key parser and an ownership heartbeat so large outputs do not accidentally expire the active session. ESC is passed to Unix applications, while Ctrl+Esc is the local detach gesture.

POC14 UX builder

The consolidated Alpine builder retained the working POC13 path for regression comparison but automated the repetitive operational setup. At boot, arm0 is configured at 10.68.0.2/30, the default route is installed through 10.68.0.1, DNS is made available, and the sysctls needed for Miami IP-NAT are applied. busybox-extras is downloaded directly from the Alpine AArch64 repository and extracted into the target rootfs without QEMU or host apk-tools.

telnetd starts automatically bound to the private ARMNET address. Sessions enter a dedicated login wrapper and receive TERM=xterm, which was specifically verified to improve the behavior of programs such as nano when accessed through MiamiTelnet. NTP runs through a retry supervisor in the background so lack of Internet connectivity at boot cannot block the service OS. /root/net-up remains only as a diagnostic fallback.

Boot diagnostics retained

The Amiga launcher now reports explicit stages for Image loading, initramfs loading, DTB/FDT work, CPU1 startup, cache synchronization, handoff, and Linux commit. Emu68 start.c exposes FLSH and HNDO status markers through the already-existing status path. A tested 1 MiB Fast-RAM staging buffer did not materially accelerate loading and was reverted. The broad 256 MiB cache flush was also retained because narrowing it did not improve the observed startup delay. The dominant UX cost remains the transfer of the roughly 23 MiB Linux Image through the AmigaOS-to-service-RAM path.

This consolidated runtime is the base on which FITZ and persistence were added today. No persistence result depends on an abandoned optimization or an unverified boot path.

4. FITZ/FUSE3 integration: Linux gains real Amiga filesystem R/W

The first major result of 31 August is verified real read/write access from Alpine Linux to AmigaOS filesystems through Fitz. The working stack is intentionally layered and conservative: Linux applications access /amiga; FUSE3 forwards operations to fitz-mount; fitz-mount communicates over TCP/ARMNET; Miami or MiamiDX supplies the Amiga bsdsocket environment; the Fitz server performs AmigaDOS filesystem operations; and AmigaOS remains the sole owner of the underlying Amiga filesystem.

```
Linux application
-> /amiga
-> FUSE3
-> fitz-mount
-> TCP over ARMNET
-> Miami/MiamiDX bsdsocket.library
-> Fitz server on AmigaOS
-> AmigaDOS filesystem
```

Kernel and initramfs requirement

The kernel had previously been configured with CONFIG_FUSE_FS=m. For this service-OS initramfs, module loading would add machinery and failure modes without providing a benefit, so FUSE was rebuilt in as CONFIG_FUSE_FS=y. The archive contains an enable-fuse-builtin.sh helper, while the freshly rebuilt kernel Image itself is not present because it was built separately and was not part of the uploaded milestone archive. ARMNET presence was checked in the rebuilt Image/vmlinux before hardware testing.

5. Storage validation through Fitz: the experiment that proved the model

Before implementing armb1k, the project tested the most important storage assumption through the already-working Fitz stack. A regular file stored on AmigaOS was exported to Linux, attached through the Linux loop layer, formatted with ext2, and exercised as a real filesystem. This deliberately inefficient path was valuable because it tested correctness across every layer before a custom block transport existed.

Deliberately not auto-mounted

/amiga/linux-loop.img (AmigaOS-owned file)

```
-> Fitz/FUSE3
-> /dev/loop0
-> ext2
-> /persist
```

The image survived block-level writes with matching SHA256 hashes. ext2 was formatted, mounted, written, synced, unmounted, detached, reattached, and mounted again without losing data. This demonstrated that AmigaOS can safely own the container file while Linux exclusively owns the filesystem structures inside it.

On real hardware, create/read/write, mkdir, rename/mv, remove, concurrent copies, and SHA256 verification were reported working. This already solves the "Linux must manipulate Amiga files" problem without any shared-filesystem corruption risk.

Why this result still matters after armb1k

The Fitz/loop arrangement is no longer the intended persistence path, but it removed the largest architectural uncertainty. It proved that the backing-file model itself is sound. /dev/armb1k0 therefore replaces only the transport layers; it does not change the ownership model that was already validated.

Direct FITZ R/W is useful on its own, but the crucial step today was to stop treating the Fitz export only as a namespace of individual Amiga files. A regular file stored on the Amiga filesystem was instead used as the backing store for a Linux loop device. This converts a safe file-level protocol into a private block device visible only to Linux, while the host still sees nothing more exotic than an ordinary file.

Verified experiment

```
/amiga/linux-loop.img (8 MiB regular file on AmigaOS)
|
+-- BusyBox losetup --> /dev/loop0
|
|   +-- mkfs.ext2
|   |
|   +-- mount --> /persist
```

The 8 MiB linux-loop.img file was created inside the Amiga filesystem through the FITZ mount. Linux attached it to /dev/loop0 with BusyBox losetup. Block-level writes through the loop device were tested

and the resulting data was compared with SHA256 hashes. The hashes matched, showing that the path remains correct not only for normal FUSE file operations but also when the file is repeatedly accessed as backing storage by the Linux block layer.

ext2 formatting and recovery

The loop device was formatted with ext2 using `mkfs.ext2 -E nodiscard -O ^inline_data`. ext2 is a sensible proof-of-concept filesystem here because it is simple, mature, does not depend on a journal, and keeps write behavior comparatively straightforward across a network-backed image file. The filesystem mounted successfully. Files were written, the filesystem was unmounted, `/dev/loop0` was detached, the same Amiga-hosted image was attached again, and the ext2 filesystem was remounted. Persistent files and SHA256 hashes were still correct after the complete detach/reattach/remount sequence.

Architectural meaning

This proves persistence at the correct ownership boundary. Linux does not mount the Amiga disk and never needs a driver for PFS3, FFS, or SFS. AmigaOS never needs to understand ext2. The only cross-OS object is `linux-loop.img`, whose bytes are transported through Fitz. Inside those bytes, Linux owns the ext2 semantics. Outside them, AmigaOS owns the containing file and host filesystem. The approach therefore avoids simultaneous filesystem ownership, the most dangerous class of designs considered earlier.

It also means persistence does not need to be tied to the physical SD layout, a dedicated partition, or a fixed PiStorm memory address. The persistent state can be copied, backed up, renamed, or moved from AmigaOS like any other file, subject only to the requirements of the Fitz export and sufficient storage.

6. Direct persistence: `armblk0.img` -> `/dev/armblk0` -> ext2

The persistence design has now moved to a dedicated block transport. The backing store is a normal file named `armblk0.img`, kept on an Amiga-accessible filesystem. The AmigaOS side owns and keeps the file open; the Linux side sees a standard block device named `/dev/armblk0`. Requests are exchanged through the existing shared service RAM rather than through TCP or FUSE.

```
apk --root /persist/apkroot --initdb --no-cache add tree
```

`armblk0.img` on AmigaOS

- > AmigaOS block backend
- > shared service RAM
- > Linux `armblk` driver
- > `/dev/armblk0`
- > ext2
- > `/persist`

For a new image, Linux can format the device with the ordinary command ``mkfs.ext2 /dev/armblk0``; the special options used during some earlier loop/FUSE experiments are not required. This is an important property of the design: once the transport has registered `/dev/armblk0` correctly, the rest of Linux should treat it as an ordinary block device.

After installation, the ext2 filesystem was unmounted, the loop device detached, and the image later reattached and remounted. The APK database, binaries, symlinks, and installed files were still present and usable. The tree executable in `/persist/apkroot/usr/bin/tree` remained intact. This is stronger evidence than merely observing that a text file survived, because package installation generates a small hierarchy with metadata and relationships that are representative of real system persistence.

Backing-file creation and EMU68BOOT

- For the public sidecARM PoC, the simplest workflow is to create a blank raw `armblk0.img` on Windows with PowerShell, then copy it to EMU68BOOT, the boot partition that Windows can access directly. A 512 MiB image can be created with `FileStream.SetLength(512MB)`. Windows must not format the image; Linux creates the ext2 filesystem on `/dev/armblk0`.
- A separate Alpine apk root created beneath `/persist/apkroot`.

- Persistent Alpine state
- The earlier Fitz-backed ext2 experiment already proved that an Alpine package root containing musl, tree, apk database files, symlinks, and binaries survives complete detach/remount cycles. armb1k now gives that same persistence model a direct transport designed specifically for block I/O. The next integration target is transparent persistent Alpine state rather than an explicit alternate apk root.
- The ext2 filesystem itself across unmount, loop detach, Fitz remount, loop reattach, and ext2 remount.

Two storage paths, two responsibilities

Fitz must remain because sidecARM needs access to the actual AmigaOS filesystem namespace. /dev/armb1k0 is intentionally different: it is private Linux storage whose contents AmigaOS treats as opaque bytes. This avoids forcing either operating system to understand or concurrently own the other's filesystem structures.

Fitz = real AmigaOS files; armb1k = private persistent Linux disk

This division is now the clearest storage architecture for sidecARM. Fitz can focus on file-level interoperability, while armb1k handles high-frequency Linux metadata, packages, configuration, and future root-overlay writes without paying the FUSE/TCP cost.

7. Measured performance and practical behavior

The Fitz validation path reached approximately 600-680 KB/s for direct sequential transfers, around 630 KB/s through the loop image, and roughly 500 KB/s for cold ext2 reads. Those measurements remain useful as a lower-level reference because the new armb1k path removes several layers that contributed to latency. The loop abstraction itself had introduced surprisingly little overhead; the larger removable costs were FUSE, TCP, Fitz protocol handling, and network traversal.

Path / operation	Observed result
Direct Fitz sequential read	~600–680 KB/s
Loop-device read	~630 KB/s
Cold ext2 read	~500 KB/s
Create + sync 500 small files	~1.15 s
Rename + sync 500 files	~1.31 s

Host-volume placement matters

A separate and important result is that Linux payload loading from SD0:/the SD boot area was approximately twice as fast as from SDH0: on the tested machine. This is why EMU68BOOT is now the preferred public-PoC location for armb1k0.img as well as the Linux payload when practical. The exact armb1k throughput should still be benchmarked independently, but host storage location is already known to be a major variable.

Small-file behavior in the Fitz-backed ext2 validation was also encouraging: 500 file creations plus sync took about 1.15 s, while 500 renames plus sync took about 1.31 s. These figures are not formal benchmarks, but they demonstrated that the backing-file model can sustain metadata-heavy workloads representative of package management and configuration state.

Windows reachability

When the Amiga forwards traffic between its normal 3Com LAN interface and ARMNET, Windows can reach the sidecARM subnet by adding a persistent route: `route -p add 10.68.0.0 mask 255.255.255.252 <AMIGA_LAN_IP>`. This is optional operational glue, not part of ARMNET itself, but it makes Linux services directly reachable from machines on the normal LAN.

Boot-time performance remains separate

The dominant visible startup cost is still loading the roughly 23 MiB Linux Image from AmigaOS into service RAM. Earlier Fast-RAM staging and narrowed cache-flush experiments did not materially

improve that delay and were reverted. The SD0/EMU68BOOT storage advantage helps, but alongside boot remains the more fundamental future solution to launcher latency.

Storage and boot performance should therefore be optimized independently: armblk simplifies persistent I/O, while future boot-path work can remove or hide the large kernel transfer.

8. Reliability model, constraints, and regression boundaries

The strongest feature of the current design is explicit ownership. AmigaOS owns armblk0.img as a normal file and services byte-range block requests. Linux owns the ext2 structures inside that image and sees only /dev/armblk0. Separately, Fitz exposes selected AmigaOS files at file level. Linux never directly mounts an Amiga block device, and AmigaOS never interprets the ext2 filesystem inside armblk0.img.

Known reliability choices

- Fitz remains outside the critical Linux persistence path; losing the Fitz server no longer means losing /persist.
- ARMNET remains the private IP transport for Linux-Amiga services and Fitz.
- armblk uses direct shared memory for block I/O and does not require TCP, FUSE, or losetup.
- armblk0.img should not be moved, replaced, or modified from AmigaOS while Linux is using /dev/armblk0.
- The Emu68 patch set remains pinned to commit 9b4379a to avoid silent source drift.

Important remaining constraints

The current service allocation remains 256 MiB, so RAM pressure on the Amiga side is still an architectural constraint. Persistence reduces the need to keep everything in the initramfs but does not remove the Linux carveout itself. The current physical-core1 IRQ workaround also remains proof-of-concept code and should eventually derive the correct hardware bank from MPIDR_EL1.

The launcher should also gain strong one-shot/liveness protection before a public release is treated as routine. The block backend must define clear behavior for Linux shutdown, sync/flush, backend failure, and stale requests. These are now the main reliability issues around persistent storage, rather than the filesystem-container model itself.

Failure behavior to preserve

A clean shutdown must issue a real block flush/sync before the AmigaOS backend closes armblk0.img. If the backing service fails unexpectedly, the Linux block device must report I/O failure rather than silently accepting writes. This is simpler than the old Fitz-backed persistence failure mode because the block service has one explicit object and one explicit responsibility.

Regression boundary for the next iteration

Do not disturb the known-good Emu68 memory allocation, ARMNET ABI, timer workaround, Fitz R/W path, or direct armblk transport without a specific reason. The next work belongs above these layers: transparent persistence, overlay/root integration, shutdown discipline, and public packaging.

9. sidecARM: public PoC and next milestone

The project now has a public-facing name: sidecARM. The intended message is concise: Linux beside the Amiga, not instead of it. The mini-PoC should ship all required tested binaries and payloads rather than asking users to rebuild Linux or reconstruct the Emu68 environment. The current baseline remains Emu68 9b4379a plus the supplied sidecARM files.

Public PoC operating model

The simplest installation path is: copy the supplied sidecARM files, create armblk0.img on Windows, copy it to EMU68BOOT, boot AmigaOS normally, run the Linux launcher, configure armnet.device in

Miami/MiamiDX at 10.68.0.1/30, and use Linux at 10.68.0.2. The existing 3Com EtherLink III remains the Amiga's normal LAN/Internet adapter in the tested topology.

Persistent storage: armb1k0.img on EMU68BOOT -> /dev/armb1k0 -> ext2 -> /persist
Amiga file access: Fitz over ARMNET -> /amiga

Recommended implementation sequence

- Stabilize direct armb1k0 READ/WRITE/FLUSH semantics and benchmark the EMU68BOOT backing file.
- Mount ext2 from /dev/armb1k0 using standard Linux filesystem tools and verify repeated cold boots.
- Move selected mutable Alpine state to /persist before attempting a whole-root overlay.
- Keep Fitz dedicated to real AmigaOS files and verify it independently of persistent Linux storage.
- Add clean shutdown/flush handling and liveness protection to the AmigaOS block backend and Linux launcher.
- Only then make normal apk add and configuration changes transparently persistent.

Definition of success for the next milestone

A convincing next milestone will boot the same immutable Alpine base, recover /dev/armb1k0 automatically, mount persistent state without Fitz being present, preserve a normally installed package and configuration change across a full Linux restart, and still allow Fitz to mount real AmigaOS files separately under /amiga.

Final status - updated 1 September 2026

sidecARM now has a clearer architecture than the original 31 August milestone. Fitz/FUSE3 remains the verified bridge to real AmigaOS files. The earlier Fitz-backed loop/ext2 experiment proved the backing-file persistence model, but private Linux persistence has moved to a purpose-built direct block device: armb1k0.img on AmigaOS is exposed as /dev/armb1k0 and formatted with ordinary ext2 semantics. ARMNET remains the private Linux-Amiga network; Miami/MiamiDX currently keeps the 3Com EtherLink III PCMCIA card as the Amiga's normal LAN/Internet interface. EMU68BOOT is the preferred Windows-accessible location for the backing image, and the SD boot area has already shown roughly twice the Linux-file loading performance of SDH0:.

Updated milestone baseline: Emu68 9b4379a plus the supplied sidecARM files; ARM64 Alpine on physical CPU1; 256 MiB service RAM; ARMNET at 10.68.0.1/10.68.0.2; Miami/MiamiDX + 3Com EtherLink III for current external networking; Fitz mounted separately for AmigaOS files; armb1k0.img on EMU68BOOT exposed directly as /dev/armb1k0 for persistent Linux storage.